Predavanja

ustni izpit: 40%

predstavitev: 20% (20 minut po 3 osebe)

Vaje

Projekt 40%

Logic in computer science

- Correctness of software/hardware/systems
  (watch Xavier Leroy: In search of software perfection)
- Developing proof assistants

Propositional logic (natural deduction proof system)
Simple type theory : Curry-Howard correspondance
(propositions as types)

Propositional logic

propositional formulas ϕ, ψ, θ

propositional variables p

atomic formula:

$$\phi ::= p \mid \psi \wedge \theta \mid \psi \vee \theta \mid \psi \to \theta \mid \neg\theta \mid \bot$$

TAVTOLOGIJA - validna formula (vedno res)

Natural deduction: proof system. Using formal rules we prove a formula ϕ

We can prove formula $\Leftrightarrow$ it is a tavtology

Primer: Proof of $p \wedge q \to p \vee r$

1. Assume $p \wedge q$ true (assumption)
2. p ($\wedge$elimination$_1$)
3. $p \vee r$ ($\vee$introduction$_1$)
4. $p \wedge q \to p \vee r$ ($\to$ introduction)

Za konstruktivno logiko (Brez LEM)

premises

$\wedge$ $\dfrac{\phi \quad \psi}{\phi \wedge \psi} \wedge i$ conclusion

$\dfrac{\phi \wedge \psi}{\phi} \wedge e_1 \qquad \dfrac{\phi \wedge \psi}{\psi} \wedge e_2$

$\vee$ $\dfrac{\phi}{\phi \vee \psi} \vee i_1 \qquad \dfrac{\psi}{\phi \vee \psi} \vee i_2$

$\dfrac{\phi \vee \psi \quad \begin{array}{c}[\phi]\\ \vdots \\ \theta\end{array} \quad \begin{array}{c}[\psi]\\ \vdots \\ \theta\end{array}}{\theta} \vee e$

$\rightarrow$ box discharges the assumption ϕ

$\dfrac{\begin{array}{c}[\phi]\\ \vdots \\ \psi\end{array}}{\phi \rightarrow \psi} \rightarrow i$

$\dfrac{\phi \rightarrow \psi \quad \phi}{\psi} \rightarrow e$ modus ponens

$\neg$ $\neg\phi \equiv \phi \rightarrow \bot$

$\dfrac{\begin{array}{c}[\phi]\\ \vdots \\ \bot\end{array}}{\neg\phi} \neg i$

$\dfrac{\neg\phi \quad \phi}{\bot} \neg e$

$\bot$ $\dfrac{\bot}{\phi} \bot e$

NO FORMUA IN THE BOX gets used outside the box.

Primer $p \wedge (q \vee r) \rightarrow (p \wedge q) \vee (p \wedge r)$

1. $p \wedge (q \vee r)$ assumption
2. p
3. $q \vee r$
4. q assumption
5. $p \wedge q$
6. $(p \wedge q) \vee (p \wedge r)$
7. r assumption
8. $p \wedge r$
9. $(p \wedge q) \vee (p \wedge r)$ $\vee e$ (3) (4-6) (7-9)
10. $(p \wedge q) \vee (p \wedge r)$ $\rightarrow i$ (1-10)

In classical logic

$\dfrac{\begin{array}{c}[\neg\phi]\\ \vdots \\ \bot\end{array}}{\phi}$ raa

Reductio ad absurdum

DN: prove LEM with raa

$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdots}}{\phi \vee \neg\phi}}{\phi \vee \neg\phi} \quad \dfrac{\dfrac{\dfrac{\dfrac{}{\neg\phi}}{\vdots}}{\bot}}{\phi}}{\phi \vee \neg\phi} \quad \dfrac{\dfrac{}{\neg\phi}}{\phi \vee \neg\phi}}{\phi \vee \neg\phi}$

A proof of formula ϕ from assumptions Γ is given by sequence of formulas following the rules such that all open assumptions in the proof are from Γ

notation: $\Gamma \vdash \phi$

Theorem: $\Gamma \vdash \phi$ (including raa) $\Leftrightarrow$ ϕ is tautology

(soundness: $\Gamma \vdash \phi \Rightarrow \phi$ is tautology; completeness: ϕ is tautology $\Rightarrow \Gamma \vdash \phi$)

$\Leftrightarrow$ ϕ is true under any assignment of truth validnes to propositional variables that make all formulas in Γ true

Formulas $\phi ::= p \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \to \phi \mid \bot$

Types $A ::= \alpha \mid A \times B \mid A + B \mid A \to B \mid$

$t : A$ — t term, A type

$$\frac{s:A \quad t:B}{(s,t):A\times B} \qquad \frac{t:A\times B}{pr_1(t):A}\times e_1 \qquad \frac{t:A\times B}{pr_2(t):B}$$

$$\frac{\begin{array}{c}[x:A]\\ \vdots \\ t:B\end{array}}{\lambda x{:}A.t : A\to B}\to i \qquad \frac{t:A\to B \quad u:A}{tu:B}\to e$$

$x:A$ variable

$$\frac{t:A}{in_1(t):A+B} \qquad \frac{t:B}{in_2(t):A+B}$$

$$\frac{s:A+B \qquad \begin{array}{|c|}\hline x:A\\ \vdots\\ t:C\\ \hline\end{array} \qquad \begin{array}{|c|}\hline y:B\\ \vdots\\ u:C\\ \hline\end{array}}{\text{case } s\,(in_1(x:A)\mapsto t \mid in_2(y:B)\mapsto u) : C}$$

Primer:

A derived term of type $A\times B \to B\times A$

1. $x : A\times B$ — assumption
2. $pr_1(x) : A$ — $\times e_1$ (1)
3. $pr_2(x) : B$ — $\times e_2$ (1)
4. $(pr_2(x), pr_1(x)) : B\times A$ — $\times i$ (3) (2)
5. $\lambda x{:}A\times B . (pr_2(x), pr_1(x)) : A\times B \to B\times A$ — $\to i$ (1-4)

tega ne potrebujemo

For every type we have introduction rules, elimination rules and computation rules

computation rule for rewrite rule (longer arrow)

$\times$: $pr_1(s,t) \longrightarrow s$

$pr_2(s,t) \longrightarrow t$

β-reduction

$\rightarrow$: $(\lambda x{:}A.t)\,u \longrightarrow t[x:=u]$

substitute u for x in t

(before doing this first rename bound variable in t that are distinct from variables in u)

$+$: $\mathsf{case}\,(in_1(s))\,(in_1(x) \mapsto t \mid in_2(y) \mapsto u) \longrightarrow t[x:=s]$

0 : $\dfrac{t:0}{\mathsf{empty}(t):A}$

DM: Find canonical term of type

$A \times (B+C) \longrightarrow A\times B + A\times C$

Propositional logic ~ Simple Type theory

Predicate logic (first order logic) ~ Dependent type theory

Propositional logic formulas

$$\phi ::= p \mid \phi \wedge \psi \mid \phi \vee \psi \mid \phi \to \psi \mid \bot$$

Predicate logic extends prop. logic with predicates (arity ≥ 0) expressing relations, and constants and function symbols (arity ≥ 0) expressing elements

Terms express elements

$$t ::= x \mid c \mid f(t, \ldots t_n)$$

(c: constant; f: function symbol of arity n)

Formulas express propositions

$$\phi ::= P(t, \ldots t_n) \mid \phi \wedge \psi \mid \phi \vee \psi \mid \phi \to \psi \mid \bot \mid \forall x. \phi \mid \exists x. \phi$$

Example: a language for expressing property of integers in predicate logic

Constants: $0, 1$

function symbols $+$, $\cdot$ of arity 2

example terms z, $x+y$, $x \cdot (y + (z+1))$

predicates $=$, $\leq$

example formulas $x \leq y$, $\exists y \exists z \exists w \exists v.\ x = yy + z \cdot z + w \cdot w + v \cdot v$

(v: free variable)

We could also define $\leq$ as

$$x \leq y \iff \exists x. \exists y. \exists v. \exists u.\ y = x + z \cdot z + w \cdot w + v \cdot v + u \cdot u$$

(vsa naravna stevila sa vsote stirih kvadratov)

$x + 1 = 0 \quad (x = -1)$

We will interpreate predicate logic in type theory

Prop ... type of propositions

$$\frac{\Phi : Prop \quad \Psi : Prop}{\Phi \wedge \Psi : Prop} \wedge \text{form.} \qquad \frac{\Phi : Prop \quad \Psi : Prop}{\Phi \vee \Psi : Prop} \vee \text{form.}$$

$$\frac{\Phi : Prop \quad \Psi : Prop}{\Phi \to \Psi : Prop} \to \text{formation} \qquad \frac{}{\bot : Prop}$$

(1) $p : Prop$ assumption
(2) $q : Prop$ assumption

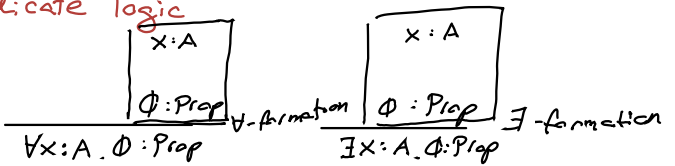
} open assumption are called a context

(3) $p \wedge q : Prop$ $\wedge$ formation (1) (2)
(4) $q \wedge p : Prop$ $\wedge$ formation (2) (1)
(5) $p \wedge q \to q \wedge p : Prop$ $\to$ formation (3) (4)

$$p : Prop, q : Prop \vdash p \wedge q \to q \wedge p : Prop$$

For predicate logic we also neen quantifiers

Predicate logic



(formation, not introduction

① $p : A \to Prop$ assumption

② $x : A$ assumption

③ $p\,x : Prop$ $\to e$

④ $\forall x : A . p\,x : Prop$ ∀-formation ② ③

$$p : A \to Prop \vdash \forall x : A . p\,x : Prop$$

(Instead of formation rules, we can include connectives and quantifiers as typed constants

$\to, \vee, \wedge : Prop \to Prop \to Prop$

$\exists, \forall : (A \to Prop) \to Prop$

iz Primera: $\forall(x : A . p(x))$

Example proof of

$p : A \to Prop,\ g : A \to Prop \vdash (\forall x : A.\ px) \to (\forall x : A.\ gx) \to (\forall x : A.\ px \wedge gx)$

3 judgments forms

$t : A$

$\Phi : Prop$ "Φ is prop."

ϕ "ϕ is true"

1. $p : A \to Prop$ assump.
2. $g : A \to Prop$ assump.
3. $\forall x : A.\ px$ assump
4. $\forall y : A.\ gy$ assump
5. $z : A$ assum
6. pz $\forall E$ (3)
7. gz $\forall E$ (4)
8. $pz \wedge gz$ $\wedge I$ (6) (7)
9. $\forall z : A.\ pz \wedge gz$ $\forall I$ (5)-(8)
10. $\forall y : A.\ gy \to \forall z : A.\ pz \wedge gz$ $\to I$ (4)-(9)
11. $\forall x : A.\ px \to (\ldots) \to (\ldots)$ $\to I$ (3)-(10)

z se ne sme pojaviti kot tekstu prej

$$\frac{\begin{array}{|c|}\hline z : A \\ \vdots \\ \Phi \\ \hline\end{array}}{\forall z : A.\Phi}\ \forall I$$

$$\frac{\forall x : A.\ \Phi \qquad t : A}{\Phi[x := t]}\ \forall E$$

Simply we replace the grammar for every type with a <u>universe</u> Type and formation rules for types

We had: $A ::= \alpha \mid A \times A \mid A + A \mid A \to A \mid 0$

$$\frac{A : Type \quad B : Type}{\begin{array}{l} A + B : Type \\ A \times B : Type \\ A \to B : Type \end{array}}$$

Key idea of dependent type theory is types depend on the context

$\mathbb{N}$ set of natural numbers
List $\mathbb{N}$ set of lists (=finite sequences) of nat. nums
Vector $\mathbb{N}$ n set of lists of n nat. numbers of length n

Dependant product

construction $\prod_{n:\mathbb{N}} \text{Vector } \mathbb{N}\ n$ | function f that map $n:\mathbb{N}$ and return a value from the set Vector $\mathbb{N}\ n$

examples

$n \mapsto \overbrace{0 \dots 0}^{n}$

$n \mapsto \underbrace{1, 2, \dots, n}_{n}$

$$\frac{\begin{array}{|l} x:A \\ \vdots \\ B:\text{Type} \end{array}}{\prod x:A.\ B:\text{Type}}\ \Pi\text{-formation}$$

type Nat

$$\frac{}{nat : Type}$$ formation rule

$$\frac{}{0 : nat} \qquad \frac{t : nat}{succ\ t : nat}$$ introduction rules

alternative

succ : nat → nat

Elimination and computation rules latter

$$\frac{t : nat \qquad P : nat \to Prop \qquad \_ : P\,0 \qquad \boxed{\begin{array}{c} x : nat \\ \vdots \\ P\,(succ\ x) \end{array}}}{P\,t}$$ Introduction rule

Instead of rule based formulations we can assume rules:

introduction: $0 : nat$ $\quad succ : nat \to nat$

elimination:

$$\forall P : nat \to Prop \left(P\,0 \to (\forall x : nat.\ P\,x \to P\,(succ\ x)) \to \forall x : nat.\ P\,x \right)$$

↑ induktaje

Iz indukcije v matematiki izrazimo recursion theorem

if v have A set and $a \in A$ and $A \to A$
then there exist unikn function $\Gamma : \mathbb{N} \to A$
da $\Gamma(0) = a$ $\Gamma(n+1) = f(\Gamma(n))$

Recursion principle

$$R_{nat} : \Pi_{P : nat \to Type} (P\,0 \to (\Pi_{x:nat}. Px \to P(succ\,x)) \to \Pi x : nat\ Px)$$

↑ recursor

$$R_{nat}\ P\ b\ f\ : \Pi_{x:nat} Px$$

computation rule:

$R_{nat}\ P\ b\ f\ 0 \longrightarrow b$

$R_{nat}\ P\ b\ f\ (succ\ n) \longrightarrow fn(R_{nat}\ P\ b\ f) : P\,succ(n)$

Example:

addition add : Nat → Nat → Nat

add := λm:nat. λn:nat.

$\mathcal{R}_{nat}$ (λ_ :nat. Nat) m (λ_ nat. λx:nat succ x) n

add $\overbrace{(\text{succ } 0)}^{m}\overbrace{(\text{succ } 0)}^{n}$ $\longrightarrow^{*}$

$\mathcal{R}_{nat}$ (_:nat. nat) succ 0 λ_:nat. λx:nat succ x) succ n

$\longrightarrow$ (λ_:nat. λx:nat succ x) 0 (($\mathcal{R}_{nat}$ λ_ nat nat)

(succ 0) (λ_:nat. λx:nat. succ x) 0))

$\longrightarrow^{*}$ succ ($\mathcal{R}_{nat}$ (λ_:nat. nat) (succ 0)

(λ_:nat. λx:nat. succ x) 0)

$\longrightarrow$ succ (succ 0) = 2 ☺

type List

$$\frac{A \text{ type}}{\text{list}A : \text{Type}} \quad \text{formation}$$

$$\frac{}{\text{nil} : \text{List}A} \qquad \frac{a : A \qquad s : \text{List}A}{\text{cons}\ s\ a : \text{list}A}$$

nil : list A

cons : A → List A → List A . alternative

elimination, computation rules

Vector type

$$\frac{A : \text{Type} \qquad n : \text{nat}}{\text{vector } A\ n : \text{Type}}$$

$$\frac{}{\text{nil} : \text{vector } A\ 0} \qquad \frac{a : A \qquad as : \text{vector } A\ n}{\text{cons}\ (n)\ a\ as : \text{vector } A\ (\text{succ } n)}$$

alternative constructs:

nil : vector A 0

cons :

$$\prod_{n:\text{nat}} A \to \text{Vector } A\ n \to \text{Vector } A\ (\text{succ } n)$$

Vector is dependent type, because it's depends on the value of n

$$\text{Rvector} : \prod_{P : \prod_{n:\text{nat}} \text{vector } A\ n \to \text{Type}} P\ 0\ \text{nil} \to \prod_{a:A} \prod_{n:\text{nat}} \prod_{as:\text{vector } A\ n} P\ n\ as \to P\ (\text{succ } n)\ (\text{cons } n\ a\ as)) \to \prod_{n:\text{nat}} \prod_{as:\text{vector } A\ n} P\ n\ as$$

Computation rules

$$P : \prod_{n:\text{nat}} (\text{vector } A\ n \to \text{Type})$$

b : P 0 nil

f :

$$\text{Rec}_{\text{vector } A}\ P\ b\ f : \prod_{n:\text{nat}} \prod_{as:\text{vec } A\ n} P\ n\ as$$

$$\text{Rec}_{\text{vec}}\ P\ b\ f\ 0\ \text{nil} \longrightarrow b$$

Rec vec P b f (succ n) (cons n

??

?

nat, List, vectors so induktivni tipi

Dependent product type

$$\frac{A \text{ type} \qquad \boxed{\begin{array}{c} x:A \\ \vdots \\ B \text{ type} \end{array}}}{\Pi_{x:A} B : \text{Type}}$$

intuitively: $\Pi_{x:A} B$ type of functions that maps $x:A$ to type B

introduction

$$\frac{\boxed{\begin{array}{c} x:A \\ \vdots \\ t:B \end{array}}}{\lambda x:A.\, t : \Pi_{x:A} B}$$

elimination

$$\frac{s : \Pi_{x:A} B \qquad t:A}{s\,t : B[t/x]}$$

Ko imamo odvisni produkt, ne potrebujemo $\rightarrow$

$A \rightarrow B := \prod_{x:A} B$, kjer B ne vsebuje x

computation rule: $(\lambda x:A.\, t)\,u \longrightarrow t[u/x]$

β-reduction

Dependent sum type

$$\frac{A : \mathrm{Type} \qquad \boxed{\begin{array}{l} x : A \\ \vdots \\ B : \mathrm{Type} \end{array}}}{\sum_{x:A} B : \mathrm{Type}}$$

$$\frac{s : A \qquad t : B[s/x]}{(s,t) : \sum_{x:A} B}$$

$$\frac{t : \sum_{x:A} B}{pr_1(t) : A} \qquad \frac{t : \sum_{x:A} B}{pr_2(t) : B[pr_1(t)/x]}$$

$$\left(A \times B :\equiv \sum_{x:A} B \right)$$

Types and their elements

0	no elements
$A \times B$	pairs (a,b), $a:A$ $b:B$
$A+B$	$in_1(a)$; $a:A$ or $in_2(b)$; $b:B$
$A \to B$	functions from A to B
$\prod_{a:A} B$	functions from $a:A \to B[a/x]$
$\sum_{a:A} B$	pairs (a,b) where $a:A$ $b:B[a/x]$

The Brower-Heyting-Kolmogorov interpretation of logic

$\bot$ has no proof

proof of $\Phi \wedge \Psi$ is pair (a,b) where a is proof of Φ and b is proof of Ψ

propositions as subterminal types

if $\Phi : Prop$ then $\Phi : Type$

($Prop$ is a subuniverse of $Type$

$Prop$ is closed under Π-type

$$\left(\frac{A\ type \qquad \left[x:A \right] \Phi\ Prop}{\Pi a:A\ B : Prop} \right)$$

if Φ is $Prop$ then Φ is subterminal

- if $a, b : \Phi$ then $a = b$ (equality next week)

Also:

$Prop : Type$ - impredicativity

We can take Π as the only constructor

Scott-Prawitz encoding of intuitionistic logic

$\bot := \Pi_{P:prop}\, P$

"for all propositions P, P is true"

$$\frac{a : \Pi_{P:prop}\, \bar{\Phi} : prop}{a\,\bar{\Phi} : \Phi} \text{ taej} \qquad \frac{\bot \quad \Phi\ prop}{\Phi}$$

$\bar{\Phi} \wedge \Psi$	$\Pi_{P:prop}\, (\bar{\Phi} \to \Psi \to P) \to P$
$\bar{\Phi} \vee \Psi$	$\Pi_{P:prop}\, (\bar{\Phi} \to P) \to (\Psi \to P) \to P$
$\Phi \to \Psi$	
$\forall x{:}A.\, \bar{\Phi}$	$\Pi_{x:A}\, \bar{\Phi}$
$\exists x{:}A.\, \bar{\Phi}$	$\Pi_{P:prop}\, (\Pi_{x:A}\, (\bar{\Phi} \to P)) \to P$

Basic properties

$A : \mathrm{Type}$

$t : A$ — needs proof that $A : \mathrm{Type}$

$s \equiv t : A$ s, t are judgmentally equal

Derived judgment forms

$\Phi : \mathrm{Prop}$ ... instance of $t : A$

$t : \Phi$

Φ means $\_ \Phi$ (can't say not $t : \Phi$)

Propositional equality

Formation

$$\frac{A \text{ type} \quad s : A \quad t : A}{s =_A t \text{ prop}}$$

$$\frac{t : A}{\text{refl}_t : t =_A t} \quad \text{intro}$$

Elim:

$$y_A \Pi_{C : (\Pi_{x:A} \Pi_{y:A} ((x =_A y) \to \text{Type}))} (\Pi_{x:A} C\, x\, x\, (\text{refl}_x)) \to \Pi_{x:A} \Pi_{y:A} \Pi_{p:(x =_A y)} C\, x\, y\, p$$

Computation:

$$y_A\, C\, f\, t\, t\, \text{refl}_{A,t} \to f\, t$$

Equality type

sodbena enakost, ne nevadna

$$\frac{u : B[s/x] \qquad s \equiv t : A}{u : B[t/s]} \text{ substitution rule}$$

$$\frac{s : A \qquad t : A \qquad s \xrightarrow{*} u \qquad t \xrightarrow{*} u}{s \equiv t : A}$$

SAT - solving (propositional logic)

$\Phi ::= x \mid \neg\Phi \mid \bot \mid \Phi \vee \Psi \mid \top \mid \Phi \wedge \Psi$ (variable)

other conectives can be defined

We view formula Φ with at most k propositional variables $x_1, \dots x_k$ as a function

$$[\![\Phi]\!] : \{0,1\}^k \longrightarrow \{0,1\}$$

([ ] semantic brackets)

- Φ in variables $x_1 \dots x_k$ is satisfiable if exists $\vec{b} \in \{0,1\}^k$, such that the formula is true on that assigment of bits
- formula is a tautology or valid if it's true for $\forall \vec{s} \in \{0,1\}^k . [\![\Phi]\!](\vec{s}) = 1$
- Φ and Ψ are ekwivalent if for $\forall b \in \{0,1\}^k . [\![\Phi]\!](\vec{b}) = [\![\Psi]\!](\vec{b})$

→ b is called satesfying assigment

Conections with the proof system

- Soundness theorem

if we can prove Φ (from no assumptions) in natural deduction, then Φ is valid.

- completeness theorem

if Φ is valid we can prove Φ in nat. deduction

(we need raa or law of excluded middle)

Φ is valid $\Leftrightarrow$ $\neg\Phi$ is not satisfiable

Definitions give rise to "truth table" algorithms, that check $\mathcal{D}$ for all $\vec{b} \in \{0,1\}^k$

Proof system prove alternative algorithm for validity: search for a proof

Negation normal form

Φ is in neg. normal form (NNF) if the only negations appear directly in front of propositional variables

$$\Phi ::= x \mid \neg x \mid \bot \mid \Phi \vee \Psi \mid \top \mid \Phi \wedge \Psi$$

Algorithm for conversion to nnf

$$\neg(\Phi \vee \Psi) \longrightarrow (\neg\Phi) \wedge (\neg\Psi)$$
$$\neg(\Phi \wedge \Psi) \longrightarrow \neg\Phi \vee \neg\Psi$$
$$\neg\neg\Phi \longrightarrow \Phi$$
$$\neg\bot \longrightarrow \top$$
$$\neg\top \longrightarrow \bot$$

(nondeterministic rewrite algorithm)

Theorem:
For any formula Φ, the rewrite algorithm terminates with the formula Φ^{nnf} that is equivalent to Φ.
Time complexity is $O(|\Phi|)$ ← size of Φ
also $|\Phi^{nnf}| = O(|\Phi|)$

A literal is a prop. variable x or the negation of x
A clause is a finite disjunction of literals
$L_1 \vee \ldots \vee L_n$ or $\bot$
(A coclause in a finite conjunction of literals
$L_1 \wedge \ldots \wedge L_n$ or $\top$)
A formula is in conjunctive normal form if it is a finite conjunction of clauses
$D_1 \wedge \ldots \wedge D_n$ or $\top$ wher D_i is a clause

A formula is in disjunctive normal form
$C_1 \vee \ldots \vee C_n$

One can give an algorithm that converts any formula Φ to Φ^{cnf} (notes)
or to Φ^{dnf}

The SAT problem

Input: is a prop. formula Φ
Output: yes if Φ is satisiable and
no if Φ is not satisfiable

The useful SAT problem

Input: is a prop. formula Φ
Output: yes if Φ satisfable + a satisfining assignment,
no if not satisfiable

Famously SAT is NP-complcate

There is no known efficient in theory algorithm

The DNF SAT problem

Input: formula in DNF
Output: yes or no

There is an easy algorithm in $O(n^2)$

The CNF sat problem (the useful one)
input: formula in CNF
output: yes or no

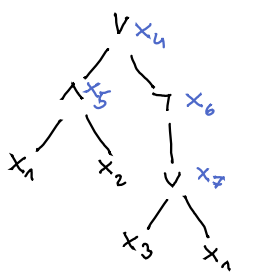
The Tseytin transformation

maps $\Phi(x_1, \dots, x_k)$ to $\Phi^T(x_1, \dots, x_K)$ where $K \geq k$ such that

- Φ^T is 3-CNF (each contains at most 3 literals)
- $(b_1, \dots, b_k) \mapsto (b_1, \dots, b_K) : \{0,1\}^k \longrightarrow \{0,1\}^K$ is a bijection satisfying assignments for Φ^T and satisfying assignments
- Φ^T is complete in $O(|\Phi|)$ time

Example

Φ: $(x_1 \wedge x_2) \vee \neg(x_3 \vee x_1)$



$x_4 \Leftrightarrow x_5 \vee x_6$

$x_5 \Leftrightarrow x_1 \wedge x_2$

$x_6 \Leftrightarrow \neg x_7$

$x_7 \Leftrightarrow x_3 \vee x_1$

implikacija $\Rightarrow$ dve implikaciji $\Leftarrow$

1.) $(\neg x_4 \vee x_5 \vee x_6) \wedge (\neg x_5 \vee x_4) \wedge (\neg x_6 \vee x_4)$

2.) $(\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_5)$

3.) $(\neg x_6 \vee \neg x_7) \wedge (x_7 \vee x_6)$

4.) $(\neg x_7 \vee x_3 \vee x_1) \wedge (\neg x_3 \vee x_7) \wedge (\neg x_1 \vee x_7)$

5.) $x_4 =$ 1) $\wedge$ 2) $\wedge$ 3) $\wedge$ 4)

Example: 3-colorability as SAT problem

Let (V, E) be a graph (undirected, irreflexive)

Construkt a formula Φ_G, with

prop variables $\{r_v, g_v, b_v \mid v \in V\}$

Φ_G in constituded as CNF formula

as a conjunction of clauses

for every $v \in V$ we include

1.) $r_v \vee g_v \vee b_v$

2) $(\neg r_v \vee \neg g_v) \wedge (\neg g_v \vee \neg b_v) \wedge (\neg r_v \vee \neg b_v)$

(vsak v ima natanko eno barvo

for every (v, v') in E we include

3) $\neg r_v \vee \neg r_{v'}$ $\neg g_v \vee \neg g_{v'}$ $\neg b_v \vee \neg b_{v'}$

Satisfying assignment for Φ_G are in 1:1 corespondece with 3-coloring of G

DPLL (Φ, ρ) ρ assigment of truth variable

Repeat until is aplicable

- if Φ is [] return ρ as the satisfying assigment
- if Φ containes ()
- if Φ conteines a unit clause L: $\rho' = \rho, L$, DPLL(simp(Φ, L), ρ')

if none of the above aply, we chose som literal L in Φ and continue with $\rho' := \rho, L$

$\Phi' =$ simplify (Φ, L). continue DPLL (Φ', ρ')

if DPLL returns ρ'' ther return ρ''

otherwise DPLL $\rho, \neg L$ and $\Phi' =$ simplify (Φ, $\neg L$)

$[(p \vee q) \wedge (\neg p \vee r) \wedge (\neg q \vee \neg r) \wedge (\neg p \vee q \vee \neg r)]$

choose p

$[r \wedge (\neg q \vee \neg r) \wedge (q \vee \neg r)]$ p

unit close r

$[\neg q \wedge q]$ p, r

unit clase $\neg q$

$[()]$ $p, r, \neg q$

consider $\neg p$

$[q \wedge (\neg q \vee \neg r)]$ $\neg p$

unit clause q

$[\neg r]$ $\neg p, q$

unti clause $\neg r$

$[]$ $\neg p, q, \neg r$

Algorithmic problems in predicate logic

In predicate logic we have a signature: a set $\mathcal{P}$ of relation symbols or predicates, a set $\mathcal{F}$ of function symbols

P has arryti ≥ 0

f has arryti ≥ 0

formulas

$$\Phi ::= P(t,\dots,t) \mid t=t \mid \neg\Phi \mid \Phi\vee\Psi \mid \Phi\wedge\Psi \mid \forall x.\Phi \mid \exists x.\Phi$$

$$t ::= x \mid f(t,\dots,t) \quad \text{terms}$$

Example signature $(\leq, 0, +)$ — 0: constant; $\leq$: relation arryti 2

Example formula: $\forall x.\ 0\leq x$

is true in natural interpretation on $\mathbb{N}$

it is false in $\mathbb{Z}$ and other interpretations in $\mathbb{N}$

A sentence is a formula with no free variables

Structures (Models)

A structure $\mathcal{M} = (M, (P^{\mathcal{M}})_{P \in \mathcal{P}}, (f^{\mathcal{M}})_{f \in \mathcal{F}})$

where M is set (domain or universe of quantification)

for every relation symbol P of arrity n

$$P^{\mathcal{M}} : M^n \longrightarrow \{\top, \bot\} = \{t, f\}$$

for every function symbol f of arrity n

$$f^{\mathcal{M}} : M^n \longrightarrow M$$

Example structure

$\mathcal{L}$

$L := \{a, 1\}^*$ ← set of finite sequences

$0^{\mathcal{L}} := \varepsilon$ ← empty sequence

$+ := w_1, w_2 \mapsto w_1 w_2$ (concatenation)

$\leq := w_1, w_2 \mapsto \begin{cases} \top & \text{if } w_1 \text{ is prefix of } w_2 \\ \bot & \text{otherwise} \end{cases}$

The sentence

$\forall x \forall y \; x+y = y+x$ holds in $\mathcal{N}$ (natural interpretation of $\mathbb{N}$), but not in $\mathcal{L}$

We write $\mathcal{N} \models \forall x \forall y \; x+y = y+x$

$\mathcal{L} \not\models \forall x \forall y \; x+y = y+x$

$\mathcal{M} \models \Phi$ .... $\mathcal{M}$ satisfies Φ

The satisfaction relation $\mathcal{M} \models_\rho \Phi$

(ρ: environment (function mapping variables to elements of M))

$\mathcal{L} \models x \leq y$ with $x \mapsto 01$, $y \mapsto 011$

$\mathcal{L} \not\models x \leq y$ with $x \mapsto 000$, $y \mapsto 0101$

$$x^\rho := \rho(x)$$
$$(f(t_1, \dots t_n))^\rho := f^{\mathcal{M}}(t_1^\rho, \dots t_n^\rho)$$

$$\mathcal{M} \models_\rho P(t_1, \dots t_n) \Leftrightarrow P^{\mathcal{M}}(t_1^\rho, \dots t_n^\rho) = +$$
$$\mathcal{M} \models_\rho t_1 = t_2 \Leftrightarrow t_1^\rho = t_2^\rho$$
$$\mathcal{M} \models_\rho \Phi_1 \wedge \Phi_2 \Leftrightarrow (\mathcal{M} \models \Phi_1) \wedge (\mathcal{M} \models \Phi_2)$$
$$\mathcal{M} \models_\rho \forall x \Phi \Leftrightarrow \text{for all } a \in M,\ \mathcal{M} \models_{\rho[x \mapsto a]} \Phi$$

Huth and Ryan textbook

A sentence Φ is valid if for all structures $\mathcal{M}$
$\mathcal{M} \models \Phi$

Soundness Theorem:
if sentence Φ has a proof in natural deduction, then Φ is valid

Completness Theorem:
if Φ is valid, then it has proof in ND.

Φ is satisfiable if there exist a structure, such that $\mathcal{M} \models \Phi$

The validity problem:
input: a sentence Φ
output: yes if Φ valid, no otherwise

Theorem (Church/Turing)
The validity problem is undecidable

A problem is decidable if there is an algorithem for it, that is garentied to terminate with the correct answer

it follows that the satisfiability problem is also undecidable

A practical algorithmic problem:
input: a sentence Φ and a puprorted ND proof of Φ in ND
output: yes if proof is correct, no otherwise
This is efficently solvable
(used in proof assistance)

Consider following algorithm for validity

Algorithm (Φ)

systematicly search the space of all proofs
until we find a proof of Φ
if we find a proof, return yes
it never returns no :

This algorithm shows that validity is semidecidable (A yes no is semidecidable if there exist an algorithm that always correctly answers yes, if the answer is yes but must forever otherwise)

Satisfiability is cosemi-decidable

cosemi-decidable means that it answers no if false, otherwise runs forever

Modeling a computer network

Signature: Connected /2 ; server, client /1

$\forall x \; \neg Conected(x,x)$

$\forall x,y. \; coneded(x,y) \rightarrow coneded(y,x)$

$\forall x. \; server(x) \vee server(x)$

$\forall x. \neg server(x) \vee \neg client(x)$

$\forall x. \; client(x) \rightarrow \exists y. \, server(y) \wedge coneded(x,y)$

$\forall x. \; server(x) \rightarrow \exists y. \; client(y) \wedge coneted(x,y)$

$\forall xy. \, coneded(x,y) \rightarrow \neg server(x) \vee \neg serve$

if consisten to have network with at least 10 devices ~~servers~~ etc that exactly 2 are ser

$\exists x_1, \dots x_{10}$

$\bigwedge_{i=1}^{9} \bigwedge_{j=i+1}^{10} x_i \neq x_j$

$\wedge \quad \exists x \exists y \; x \neq y \wedge \forall z. \; server(z) \rightarrow z = x \vee z = y)$

$\wedge \; server(x) \wedge server(y)$

Finite satisfiability

Question: sentence Φ

Answer yes if exists finite structure $\mathcal{A}$, such that $\mathcal{A} \models \Phi$ and no otherwise

Finite satisfiability is semidecidable

Algorithm: for every size $n = 1, 2, \ldots$ we enumerate all non-isomorphic structures of size n and for each structure $\mathcal{A}$ we look at the model checking problem $\mathcal{A} \models \Phi$, if and when we find such a structure, we return that structure

24.3 nissim pisela

SMT-solving

Satisfy modulo a theory

SAT	prop. logic	not very expressive but solving is practical
FOL	...	good expressivity but solving is impractical (but not entirely)

SMT — the sweetspot in the middle. Expressive enough and satisfiability is still practical

SMT basic setting:

First order signature: function symbols + predicates

Tipicly we are interested in the question:

Is Φ satisfiable modulo a theory T?

For our purposes a theory T is determined by a set of structures mod_T

We are interested in the question

$\exists \mathcal{M} \in \text{Mod}_T.\ \exists \text{environment } \beta.\ \mathcal{M} \models_\beta \Phi$?

SMT is useful for program verification.
Construct $\Phi(x_1, \ldots, x_n)$ saying $x_1, \ldots x_n$ are inputs on which the program behaves incorrectly.
A satisfying assignment to Φ finds a bug in input.
If unsatisfiable, there are no bugs

Interesting theories:

EUF: equality between uninterpreted functions
signature: just function symbols
Flatten formula: no nesting of function symbols, every term named by a variable
Perform congruence closure
Check consistently of the resulting congruence closure with negated equality

$x=y \wedge x = g(y) \wedge u = f(x, g(x)) \wedge v = f(y, x) \wedge u \neq v$

$x=y \wedge x = g(y) \wedge t = g(x) \wedge u = f(x,t) \wedge v = f(y,x) \wedge u \neq v$

$[x|y|t|u|v]$ ... partition of all variables into separate equivalence classes

$[x,y|t|u|v]$

$[x,y,t|u|v]$

$[x,y,t|u,v]$ $\Rightarrow$ formula is unsatisfiable

- Have algorithm congruence closure decide satisfiability for coclauses (conjunctions of literals) (practical)
- So we can decide satisfiability arbitrary quantifier free formulas by reduction to DNF (impractical)

Anja Petković

Theorry of arrays:

Two functions: write(a, i, v), read(a, i)

Theory:
$i = j \rightarrow read(a,i) = read(a,j)$
$i = j \rightarrow read(write(a,i,v),j) = v$
$i \neq j \rightarrow read(write(a,i,v),j) = read(a,j)$

Reducing array to EUF

$i \neq j \land u = read(write(a,i,v),j) \land v = read(a,j) \land u \neq v$

$i \neq j \land u = (i = j \ ? \ v : read(a,j)) \land v = read(a,j) \land u \neq v$

↑ ite ↑ pattern ↑ slicer

$i \neq j \land u = read(a,j) \land v = read(a,j) \land u \neq v$

$[i | j | u | v]$ Now we preform satisfiability test

$[i | j | u, v]$ unsatisfiable!

What we aim for with SMT?
We have a theory, that supports efficient satisfiability for conjunctions. We want to bootstrap to arbitrary quantifier free formulas

2 approaches:
- Eager: Translate to a propositional sat problem
- Lazy: Modify SAT solving algorithm based on DPLL. To find contradiction we use conjunction decision procedure

→:

start with a flatten formula

$$x=f(y) \wedge y=f(x) \wedge x=y$$

replace terms with variables

$$x=f^y \wedge y=f^x \wedge x=y \wedge (x=y \rightarrow f^x=f^y)$$

We obtain in the end a first order formula, where all atomic formulas are equalities between variables

Let k be the smallest number, such that $2^k \geq$ number of variables.
Bit blasting: replace every variable x with k propositional variables $x_1, \dots, x_k$
Replace every equality $x=y$ with

$$x_1 \leftrightarrow y_1 \wedge x_2 \leftrightarrow y_2 \wedge \dots \wedge x_k \leftrightarrow y_k$$

$x \neq y$ with $\neg(\dots)$
we submit to SAT solver.
This will be satisfiable $\Leftrightarrow$ it's unsatisfiable

start with flatten formula $x=f(y)$, $y=f(x)$, $x \neq y$

$$x=f^y \wedge y=f^x \wedge x \neq y \wedge (x=y \rightarrow f^x=f^y)$$

$k=2$

$$(x_1 \leftrightarrow f_1^y) \wedge (x_2 \leftrightarrow f_2^y) \wedge (y_1 \leftrightarrow f_1^x) \wedge (y_2 \leftrightarrow f_2^x) \wedge$$
$$\neg(x_1 \leftrightarrow y_1 \wedge x_2 \leftrightarrow y_2) \wedge ((x_1 \leftrightarrow y_1 \wedge x_2 \leftrightarrow y_2) \rightarrow f_1^y \leftrightarrow f_1^x \wedge f_2^y \leftrightarrow f_2^x$$

linear arithmetic

terms 0 $+$ 1 $a \cdot x$

↑ integer / natural / real / rational ....

$\leq$

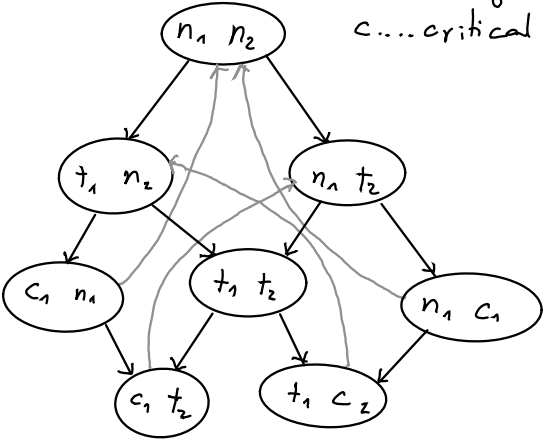
Model Checking

Today: Linear Time Temporal Logic (LTL)

Further reading: HR, chapter 3

Example of transition system:

n ... noncritical
t ... trying
c ... critical



Example safety property (the system always stays in safe state

globalno ← 1 in 2 nista critical na enkrat

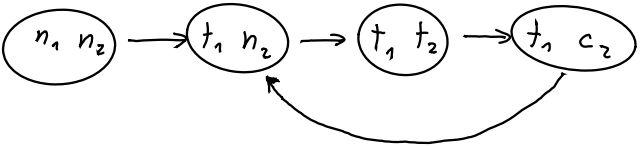
$G(\neg(c_1 \wedge c_2))$ trivialy holds

We know also liveness property (something desirable will eventually happen)

$G((t_1 \rightarrow F\, c_1) \wedge (t_2 \rightarrow F\, c_2))$

(če 1 poskuša dobiti dostop, ga bo enkrat dobila)

This does not hold. Counter run:



This is called „lasso" run

A transition system is given by

$(S, \rightarrow, L, Atoms)$

where S is finite set (of states)

$\rightarrow \subseteq S \times S$ (the transition relation)

$L : S \rightarrow \mathcal{P}(Atoms)$ maps state to it's atoms

The labeling function — Powerset of atoms

$Atoms = \{n_1, n_2, t_1, t_2, c_1, c_2\}$

LTL formulas:

$\Phi ::= p \mid \neg\Phi \mid \Phi \wedge \Phi \mid \Phi \vee \Phi \mid X\Phi \mid F\Phi \mid G\Phi \mid \Phi \mathcal{U} \Psi$

p: Atoms

$X\Phi$: Φ holds in next

$F\Phi$: Φ holds in future

$G\Phi$: Φ holds globally

$\Phi \mathcal{U} \Psi$: now and in the future Φ holds until Ψ holds

in textbook also $\mid \Phi \mathcal{R} \Psi \mid \Phi W \Psi$

Requements for LTL

$\forall s \in S. \exists s' \in S. s \rightarrow s'$

The satisfaction relation take the form

$$\pi \models \phi$$

Here π is infinite run trought transition systems

$$\pi = s_1 \to s_2 \to s_3 \to \dots$$

satisfaction relation

$$\pi \models p \iff p \in L(s_1) \quad (p \in L(\pi(1)))$$

$$\pi \models \neg\phi \iff \pi \not\models \phi$$

$$\pi \models \phi \wedge \psi \iff \pi \models \phi \wedge \pi \models \psi$$

$$\pi \models X\phi \iff \pi^2 \models \phi$$

τ ϕ holds in run started on second

$$\pi \models G\phi \iff \forall n.\, \pi^n \models \phi$$

$$\pi \models F\phi \iff \exists n.\, \pi^n \models \phi$$

$$\pi \models \phi \,\mathcal{U}\, \psi \iff \exists n.\, \pi^n \models \psi \wedge \forall m < n.\, \pi^m \models \phi$$

$$\pi^n := s_n \to s_{n+1} \to \dots$$

Model checking problem

Input: A transition system $\mathcal{M}$
A system state s_0
An LTL formula Φ

Output:
yes, if $\forall$ runs π through $\mathcal{M}$ starting at s_0, we have $\pi \models \Phi$ $\quad \mathcal{M}, s_0 \models \Phi$
No, since $\exists$ an infinite run π that fails

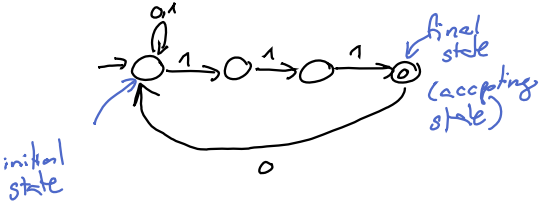
The liveness property for mutual exclusion
$G((t_1 \rightarrow F c_1) \wedge t_2 \rightarrow F c_2))$

DM: modify the model where this holds

Alternative: Assume fairness property
A fairness assumption:
$\Phi_{fairns} = GF(t_1 \wedge t_2) \rightarrow GF(c_1) \wedge GF(c_2$

Model checking formula: $\Phi_{fairness} \rightarrow \Phi$ liviness

Nondeterministic Büchi automata



Alphabet: $\{0,1\}$

A büchi automata defines a language of infinite words

The set of infinite words is written Σ^ω

Examples of ω-words

0 0 0 0 ...
0 1 0 1 ...
⋮

infinite word is accepted, if there is some run of the automata, labeled by the word, that passes accepting state infinetly many times

Model checking problem for LTL

input: Transition system $\mathcal{M}$, starting state S, LTL formula Φ

Output: yes, if $\mathcal{M}, S \models \Phi$,
no, otherwise + lasso run π such that $\pi \models \neg\Phi$

Language theory alphabet Σ, word is a finite sequence from Σ (Σ^* for the set of words)

A langue is a set of words, $L \subseteq \Sigma^*$

Language theory for ω-words

ω-word is an infinite segence from Σ (Σ^ω for the set of ω-words)

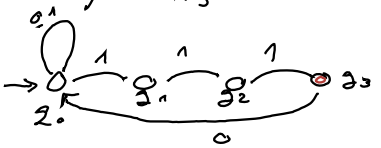
An ω-language is a set of ω-words $L \subseteq \Sigma^\omega$

An automate on ω-words specifies an ω-language

Büchi automata is a particular automata on ω-words

$\Sigma = \{0,1\}$ A_1 will specify:

$L_\omega(A_1) = \{ w \in \{0,1\}^\omega$; w contains substring 1110 infinetly often $\}$



A_2:

$L_\omega(A_2) = \{ w \in \{0,1\}^\omega$ | from some point there are only ones. There are finitly many zeros$\}$

A_3:

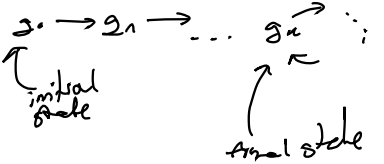
$L_\omega(A_3) = \{\}$

A Büchi automata is formally a tuple (Q, I, Δ, F) A

- Q: set of states
- I: set of initial states $I \subseteq Q$
- F: set of final states $\Delta \subseteq Q$
- Δ: Transition relation $\Delta \subseteq Q \times \Sigma \times Q$

nondeterministic Büchi automata

Proposition (Emptiness testing for an NBA)
Let A be an NBA
$L_\omega(A) \neq \emptyset \Leftrightarrow$ there exists a path of the form



Such a path is called an accepted lasso

Definition: An ω-language $L \subseteq \Sigma^\omega$ is ω-regular if there exists NBA A such that $L_\omega(A) = L$

Theorem: ω-regular languages are closed
- under union
- intersection
- complement (Büchi's theorem)

Consider an LTL formula φ
Consider when does it hold that a run satisfies

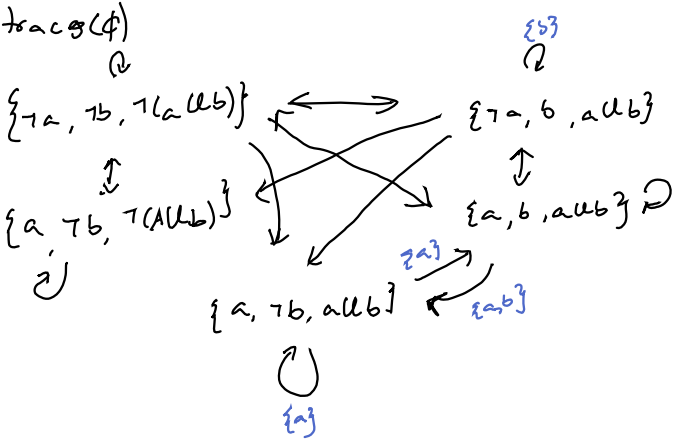
$\pi \models \varphi$

$\pi = s_1 \to s_2 \to s_3 \to \dots$

The Trace of a run $\pi = s_1 \to s_2 \to \dots$
is the sequence $L(s_1), L(s_2), L(s_3) \dots$ (← label function)
This is an ω-word in $\mathcal{P}(\text{Atoms})^\omega$
The truth of a formula φ on a run π is completely determined on the trace

$\text{Traces}(\varphi) := \{ w \in \mathcal{P}(\text{Atoms})^\omega \; ; \; \varphi \text{ is true on any run with the } w \}$

Theorem: For any ϕ, Traces(ϕ) is ω-regular.
There is an algorithm to construct A_ϕ recognising

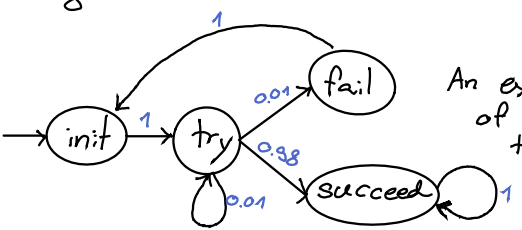


Model Checking algorithm
st 1 construct the automaton $A_{\neg\phi}$
st 2: $A_{\neg\phi} \otimes M$

Probabilistic Model Checking

In general:

- many probabilistic models (today we focus on DTMCs ← discrete time Markov chain)
- many probabilistic logics (PCTL, PCTL*) (we are going to use LTL)
- There is a whole technology of associated algorithms



An example of discrete time Markov chain

$P(init) = 1 \quad P(X\, try) = 1 \quad P(XX\, succ) = 0.98$

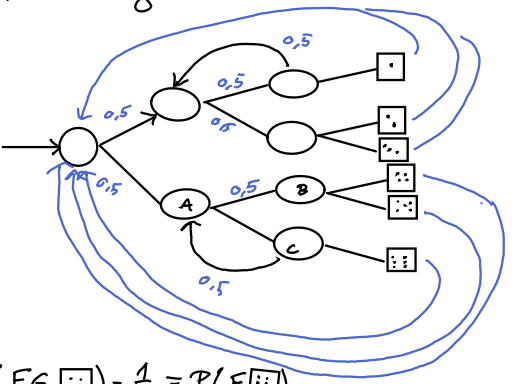
$P(X(try \wedge X(try \wedge X\, try))) =$

$= P(XX(try \wedge X\, try)) = P(XXX\, try) = 0.0001$

$P(FG\, succ) = 1 - (0,01)^n (0,01)^m \xrightarrow{n,m \to \infty} 1$

n: τ_{try}, m: τ_{fail}

Knuth/Yao-algorithem for a fair dice



$P(FG\,\boxdot) = \frac{1}{6} = P(F\,\boxdot)$

modified: $P(\text{ou}\,\boxdot) = \frac{1}{6}$

$P(\text{ou}\,(\boxdot \wedge \text{x ou}\,\boxdot)) = \frac{1}{36}$

$P_B = 0 \qquad P_C = \frac{1}{2} + \frac{1}{2} P_A \qquad P_A = \frac{1}{2} P_B + \frac{1}{2} P_C =$

$= \frac{1}{4} + \frac{1}{4} P_A \quad \rightsquigarrow \quad P_A = \frac{1}{3}$

General solution:

We have DTMC and LTL formula Φ

Question: What is the probability that a randomly generated run π satisfies Φ

Two issues:

1. Make sense of the question
2. Compute the answers

A DTMC is (S, s_{init}, P, L) — P: transition matrix; S: set of states; L: labeling function

- S set of states (in modeling is normaly finite)
- $s_{init} \in S$ the initial state
- L labeling function $: S \to \mathcal{P}(Atoms)$
- $P : S \times S \to [0,1]$ transition probability matrix satisfying $\forall s \in S. \sum_{t \in S} P(s,t) = 1$

for a formula Φ we want to calculate

$$\mathbb{P}[\Phi] := \mathcal{P}(\{\pi \in Runs \mid \pi \models \Phi\})$$

We assign probability to measurable sets of runs

A cylinder set is determined by a finite run

$w = s_1 \xrightarrow{p_1} s_2 \xrightarrow{p_2} \dots \to s_n \qquad s_1 = s_{init} \qquad p_i > 0$

$$\langle w \rangle := \{\pi \in Runs \mid \pi \text{ extends } w \ (w \text{ is prefix of } \pi)\}$$

$$\mathbb{P}\langle w \rangle = \prod_{i=1}^{n-1} p_i$$

The measurable subset of Runs are defined by

- Every cylinder set $\langle w \rangle$ is measurable
- If $R_1, R_2, \dots$ are measurable sets then so are $\bigcup_n R_n$, $\bigcap_n R_n$
- If R is measurable, then so is R^c
- $\emptyset$ and Runs are measurable

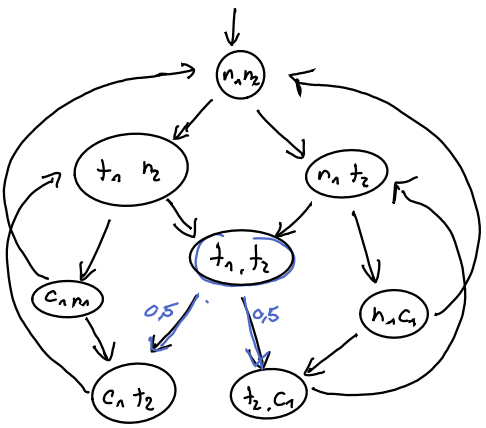
A general fact: Let $\mathcal{E}$ for the set of all measurable subset of Runs. There is a unique probability function $\mathbb{P}: \mathcal{E} \to [0,1]$ satisfying

$\mathbb{P}$ is a probability measure:
- $\mathbb{P}\emptyset = 0$
- $\mathbb{P}\langle w \rangle$ is as defined earlier
- If we have $R_1, R_2, \ldots$ pairwise disjoint measurable sets then $\mathbb{P}(\bigcup_n R_n) = \sum_n \mathbb{P} R_n$ (Countable additivity)

Proposition: For any formula Φ, the set $\{\pi \in Runs \mid \pi \models \Phi\}$ is measurable.

The proof uses Büchi automata

Model checking: Given (S, s_{init}, P, L) and Φ answer $P_{>g}\Phi$ = is $\mathbb{P}(\Phi) > g$



This is a Markov decision process (MDP) is combining probability and determinism

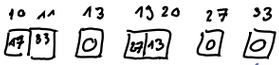
Liveness: $G((t_1 \to F c_1) \wedge (t_2 \to F c_2))$

A Markov run would be $(n_1, n_2) \to n(t_1)$ $\xrightarrow{0,5}$ $(c_1, t_2) \to \ldots$ and $\xrightarrow{0,5}$ $(c_2, t_1) \to \ldots$

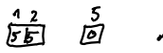
livness: $G((t_1 \to F c_1) \wedge (t_2 \to F c_2))$ has probability 1 over all markov chains

misli: na da sem
manjkala 28.4

Binary tries (linked trees in heap memory)



↑ to predstavlja list



tega ne bomo dovolili:

ltree(t, i) .... linked tree. i is a reference to a linked binary tree in heap memory and the implemented tree is the matematical tree t

ltree(nd(nd(lf, lf), lf), 10)

↑ kje je root

¬ltree(nd(lf, lf), 1)

ltree is defined inductiuly on the structure of the mathematical tree t and moreover, the implemented tree is the only defined part of heap memory

$$ltree(lf, i) \iff i \mapsto 0$$

$$ltree(nd(t_1, t_2), i) \iff \exists j,k.\ (i \mapsto j) * (i+1 \mapsto k) * ltree(t_1, j) * ltree(t_2, k)$$

Program: gre čez drevo in ga izbriše

$\{\exists t.\ tree(t,i)\}$

```
def delTree (i)
    if [i]=0 :
        dispose (i)
    else:
        j := [i]
        k := [i+1]
        delTree (j)
        delTree (k)
        dispose (i)
        dispose (i+1)
```

$\{\exists t.\ ltree(t,i) \wedge [i]=0\}$

$\{i \mapsto 0\}$ implied

$\{emp\}$

$\{\exists t.\ ltree(t_1;) \wedge [i] \neq 0\}$

$\{\exists j,k.\ [i]=j * [i+1]=k * ltree(t_1, j) * ltree(t_2, k)$

$\{i \mapsto j * i+1 \mapsto k * ltree(t_1, j) * ltree(t_2, k)\}$

$\{i \mapsto j * i+1 \mapsto k * emp * ltree(t_2, k)\}$

$\{i \mapsto j * i+1 \mapsto k * emp * emp\}$

$\{emp * emp * emp * emp\}$

$\{emp\}$

$\{emp\}$

$s, h \vDash emp \iff h = \emptyset$

small rule: $\dfrac{}{\{n \mapsto \_\}\ dispose(n)\ \{emp\}}$

$$\frac{\{\phi[E_1/i]\}\ proc(E_1)\ \{\psi[E_1/i]\} \quad \{\phi[E_2/i]\}\ proc(E_2)\ \{\psi[E_2/i]\} \quad \vdots}{\{\phi\}\ C\ \{\psi\}}$$

$$\{\phi\}\ proc(i) : C\ \{\psi\}$$

$$\frac{\{\phi\}\ C\ \{\psi\}}{\{\phi * \theta\}\ C\ \{\psi * \theta\}}\ FV(\theta) \wedge MV(C) = \emptyset$$

↑ free variables of θ

↑ set of variables that C modifies

Machine configuration: state + heap

state: function

heap: finite partial function $\mathbb{N} \rightharpoonup \mathbb{Z}$

Assertions ϕ state property of configurations with $s, h \models \phi$ (s, h asserts ϕ)

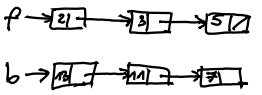
$s, h \models n \mapsto v \iff h = \{(n, v)\}$ (partial function (as a graph))

$s, h \models \phi * \psi \iff \exists h_1, h_2.\ h = h_1 \oplus h_2$ and $s, h_1 \models \phi$ and $s, h_2 \models \psi$ ($\oplus$: disjoint union)

Resource analysis with seperation logic

Amortised time analysis:

Functional queues



Makequ: $O(1)$
f := 0
b := 0

Enqu (x)
i := allocate(2)
[i] := x
[i+1] := b
b := i

nejde v spanu:
Domo na konec vrste

Deque
if f = 0:
then reverse (b) and put it in f $O(n)$
b := 0
else: skip

if f = 0 : (if it is still zero?):
then error
else v := [f];
k := [f+1]
dispose f ← deborama spanit
f = k
return v

Dequ: $O(1)$ if $f \neq 0$
$O(n)$ if $f = 0$

$\frac{T_{total}}{N} \leq O(1)$ for every N

Extended seperation logic with resource credits

$\{1\epsilon\}$ Makequ $\{queue(f,b)\}$

$queue(f,b) \Leftrightarrow flist(f) * blist(b)$

$flist(f) :\Leftrightarrow ((f=0 \wedge emp \wedge 0\epsilon) \vee (f>0 \wedge \exists v, g\ [f \mapsto v] * [f+1 \hookrightarrow g] * flist(g)) \wedge 0\epsilon$

$blist(b) :\Leftrightarrow (b=0 \wedge emp \wedge 0\epsilon) \vee (b>0$ b b+1 $blist(g) * 1\epsilon$

$\{2\epsilon * queue(f,b)\}$ Enqueue $\{queue(f,b)\}$

$\{1\epsilon * queue(f,b)) \wedge \neg(f=b=0)\}$ Dequ $\{queue(f,b)\}$

$\{f = 0 \land \text{blist}\, b\}$

$\{\text{flist}(f) * \text{blist}(b)\}$

while $b \neq 0$

$\{(\text{flist}(f) * \text{blist}(b)) \land b \neq 0\}$

$\{\text{flist}(f) * \exists v, g.\ [b \mapsto v] * [b+1 \mapsto g] * 1\epsilon * \text{blist}(g)\}$

tick

$\{\text{first}(f) * \exists v, g\ [b \mapsto v] * [b+1 \mapsto g] * \text{blist}(g)\}$

$k := [b+1]$

$\{\exists v.\ \text{flist}(f) * [b \mapsto v] * [b+1 \mapsto f] * \text{blist}(k)\}$

$\{\text{flist}(b) * \text{blist}(k)\}$ ← implies

$f := b$

$\{\exists v.\ \text{first}(f) * \text{blist}(k)$

$b := k$

$\{\text{flist}(f) * \text{blist}(b)\}$

$\{b = 0 \land (\text{flist}(f) * \text{blist}(b))\}$

$\{\text{flist}(f) \land b = 0\}$

Soundness and completeness of Hoare logic

Hoare triple: $\{\phi\}\, c\, \{\psi\}$ (c: comands)

$c ::= x := E \mid \text{if } B \text{ then } C \text{ else } C \mid c ; c \mid \text{skip} \mid \text{while } B \text{ do } C$

$E ::= n \mid x \mid -E \mid E + E \mid E * E \mid \ldots$

$B ::= E = E \mid E < E \mid B \vee B \mid B \wedge B \mid \neg B$

$\vdash_{par} \{\phi\}\, c\, \{\psi\} \iff$ there exists a proof of the specification $\{\phi\} c \{\psi\}$

For any command C and postcondition ψ,
we define $wp(C, \psi)$ by:

$wp(C, \psi) \Leftrightarrow$ if the execution of program C terminates, then the resulting state satisfies ψ

Lemma (weakest precondition)

1. $\vdash_{par} \{wp(C, \psi)\}\ C\ \{\psi\}$
2. $\vdash_{par} \{\phi\}\ C\ \{\psi\} \Rightarrow \mathbb{Z} \models \phi \to wp(C, \psi)$

Lemma (expressive completeness)
The property $wp(C, \psi)$ can be explained by a formula in our assertion logic

Very quick outline of proof:
Prove by induction on C that $\vdash_{par} \{\phi\}\ C\ \{\psi\}$
$\Rightarrow \vdash_{par} \{\phi\}\ C\ \{\psi\}$.
Eg, in the case of while B do C.
Suppose $\vdash \{\phi\}$ while B do C $\{\psi\}$

Oral exams

- No type theory

present for 10 minutes
← choose
easy and hard topics

(10) · LTL specification for DTMCs
(≈ 13) · seperation logic
(last) · tc completness theorem for Hoare logic

- SAT solving and DPLL algorithm
- Predicate logic and bounded model checking
- SMT solving
(· LTL for transition systems) - exaled for me
- Hoare logic

Sign up for exams

8.~~6~~ - kon ec. ~~6~~